

Reviewer Pack — Algebraic Shifting Generator Count Bounds Communication Comp...

Entry 1c840e93cd98 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 09:31:54 UTC

Algebraic Shifting Generator Count Bounds Communication Complexity

Entry ID: 1c840e93cd98 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-13 09:31:54 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Commutative Algebra (Algebraic Shifting) **Field B** (complexity object): Communication Complexity of Boolean Functions

Statement:

For a Boolean function f on n variables, the communication complexity $CC(f)$ is at least the number of minimal generators of the ideal defining the algebraically shifted simplicial complex of f 's clauses.

Rationale (proposer's reasoning):

Algebraic shifting captures the minimal structure of a simplicial complex, which might reflect the minimal communication required to compute the function, linking combinatorial algebra to communication complexity.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 98b29906268746e1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random CNF formula with n variables and m clauses
    n = 10
    m = 2 * n
    cnf = []
    for _ in range(m):
        clause = [random.randint(1, n), -random.randint(1, n)]
        cnf.append(clause)

    # Convert CNF to simplicial complex (simplified representation)
    simplicial_complex = {tuple(sorted(cnf[i])) for i in range(m)}

    # Apply algebraic shifting (simplified version)
    shifted_complex = set()
    for face in simplicial_complex:
        if len(face) > 1:
            for i in range(len(face)):
                new_face = list(face)
                new_face.remove(-face[i])
                shifted_complex.add(tuple(sorted(new_face)))

    # Compute the ideal's generators (simplified version)
    generators = []
    for face in shifted_complex:
        if len(face) == 1:
            generators.append((1, -face[0]))
        else:
            generators.append((-1, *face))

    # Communication complexity (example protocol: one-bit communication per variable)
    cc = n

    # Count the number of minimal generators
    num_generators = len(generators)
```

```

# Check if the conjecture holds
conjecture_holds = num_generators >= cc

return {
    "metric_name": "communication_complexity",
    "metric_value": cc,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"CNF: {cnf}, CC: {cc}, Generators: {generators}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 53)) # Default to first 30 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_cc = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_cc) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_cc} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a4b6e23.py", line 72, in <module>

```

    result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2a4b6e23.py", line 38, in run_trial

```

    new_face.remove(-face[i])

```

ValueError: list.remove(x): x not in list

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before collecting data, preventing evaluation of support fraction or counterexamples. | next: Fix the list operation error in the test code and re-run with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	110932
2	preregistration	ollama_remote	qwen3:8b	0	0	31190
3	novelty	ollama_remote	qwen3:8b	0	0	24946
4	novelty	ollama_remote	qwen3:8b	0	0	16411
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18303
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7822
7	judge	ollama_remote	qwen3:8b	0	0	13972

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 223576 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/1c840e93cd98.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/1c840e93cd98.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/1c840e93cd98.tar.gz (if generated)